simple-plot

Typst Package

Mathematical Function Plotting

A lightweight library for creating elegant mathematical plots

Version 0.9.0

Nathan Scheinmann

Contents

1. Introduction

`simple-plot` is a Typst package for creating clean, elegant mathematical plots. Built on CeTZ, it provides an intuitive interface for plotting functions, data points, and creating publication-ready graphs.

1.1. Features

- Plot mathematical functions with automatic sampling
- Parametric curves (ellipses, hyperbolas, closed shapes)
- Scatter plots and line plots with customizable markers
- Fill areas under curves or between two curves (solid or hatched)
- Clean integer-based tick system by default
- Major and minor grid with elegant styling
- Gap-based grid line breaks around tick labels (grid-label-break)
- Automatic axis extension beyond grid
- Flexible axis positioning (origin, bottom/left, custom)
- Multiple label display options (unit-label-only, label-step)
- Function labels with flexible positioning
- Text annotations and reference lines (vline, hline)
- Riemann sum rectangles with left, right, midpoint, lower, and upper methods
- Volume of revolution diagrams with end caps, disks, and tilted axes
- Clipping for clean rendering at boundaries
- Global defaults and style overrides

1.2. Installation

Import the package in your Typst document:

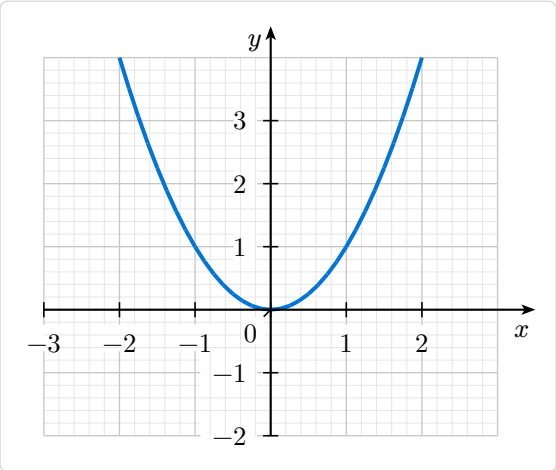
```
#import "@preview/simple-plot:0.9.0": plot
```

1.3. Quick Start

Code

```
#plot(
  width: 6, height: 5,
  xmin: -3, xmax: 3, ymin: -2, ymax: 4,
  xlabel: $x$, ylabel: $y$,
  show-grid: true,
  (fn: x => x * x, stroke: blue + 1.5pt),
)
```

Preview



2. Basic Usage

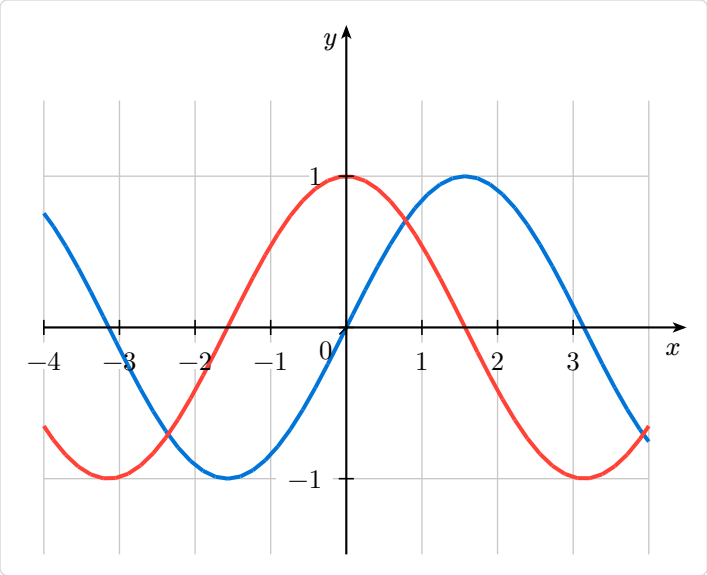
2.1. Plotting Functions

Plot mathematical functions by passing a dictionary with `fn`:

Code

```
#plot(
  width: 8, height: 6,
  xmin: -4, xmax: 4, ymin: -1.5, ymax: 1.5,
  xlabel: $x$, ylabel: $y$,
  show-grid: "major",
  (fn: x => calc.sin(x), stroke: blue + 1.5pt),
  (fn: x => calc.cos(x), stroke: red + 1.5pt),
)
```

Preview



2.2. Mathematical Functions Reference

Functions are defined using Typst's `calc` module. Here are the most common mathematical functions:

Function	Typst syntax
Power x^n	`calc.pow(x, n)`
Square root $\sqrt{x}$	`calc.sqrt(x)`
Absolute value $\|x\|$	`calc.abs(x)`
Sine $\sin(x)$	`calc.sin(x)`
Cosine $\cos(x)$	`calc.cos(x)`
Tangent $\tan(x)$	`calc.tan(x)`
Exponential e^x	`calc.exp(x)`
Natural log $\ln(x)$	`calc.ln(x)`
Log base b	`calc.log(x, base: b)`
Maximum	`calc.max(a, b)`
Minimum	`calc.min(a, b)`

Important: When using constants in calculations, use decimal notation (e.g., `2.0` instead of `2`) to avoid type errors. For example:

- ✓ `x => x * x / 2.0`
- ✗ `x => x * x / 2` *(may cause errors)*

This is because Typst's type system requires consistent float arithmetic.

To draw a function with a hole or discontinuity, return `none` from the function at that point — the line will break without connecting across the gap:

```
// Draws 1/x with a break at x=0
(fn: x => if calc.abs(x) < 0.01 { none } else { 1.0 / x }, stroke: blue + 1.5pt)
```

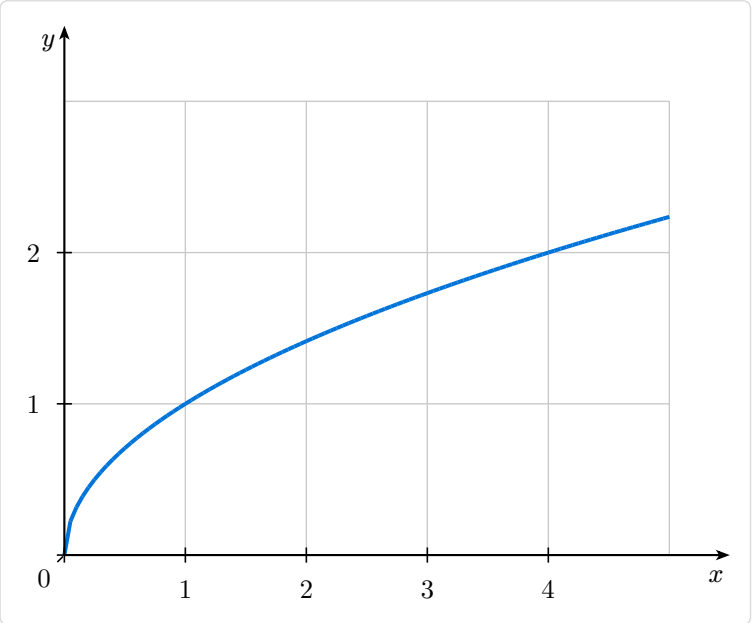
2.3. Function Domain

Specify a custom domain for functions:

Code

```
#plot(
  width: 8, height: 6,
  xmin: 0, xmax: 5, ymin: 0, ymax: 3,
  xlabel: $x$, ylabel: $y$,
  axis-x-pos: "bottom", axis-y-pos: "left",
  show-grid: "major",
  (fn: x => calc.sqrt(x), domain: (0, 5), stroke: blue + 1.5pt),
)
```

Preview



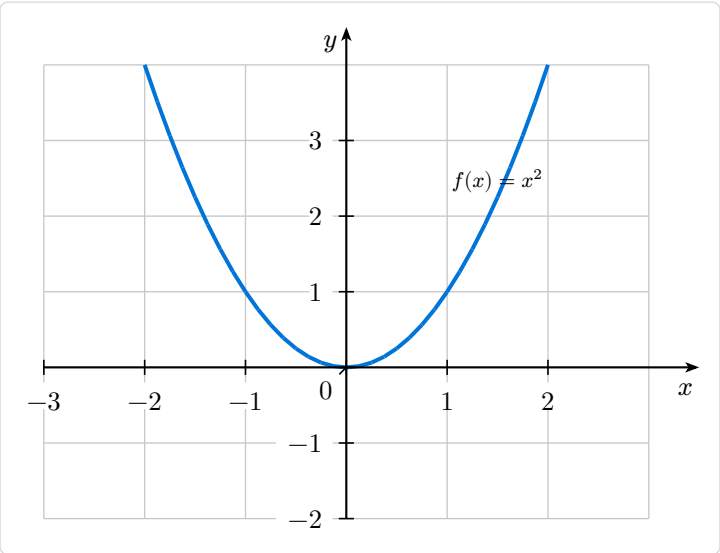
2.4. Function Labels

Add labels to your functions:

Code

```
#plot(
  width: 8, height: 6,
  xmin: -3, xmax: 3, ymin: -2, ymax: 4,
  xlabel: $x$, ylabel: $y$,
  show-grid: "major",
  (
    fn: x => x * x,
    stroke: blue + 1.5pt,
    label: $f(x) = x^2$,
    label-side: "above",
    label-pos: 0.75,
  ),
)
```

Preview



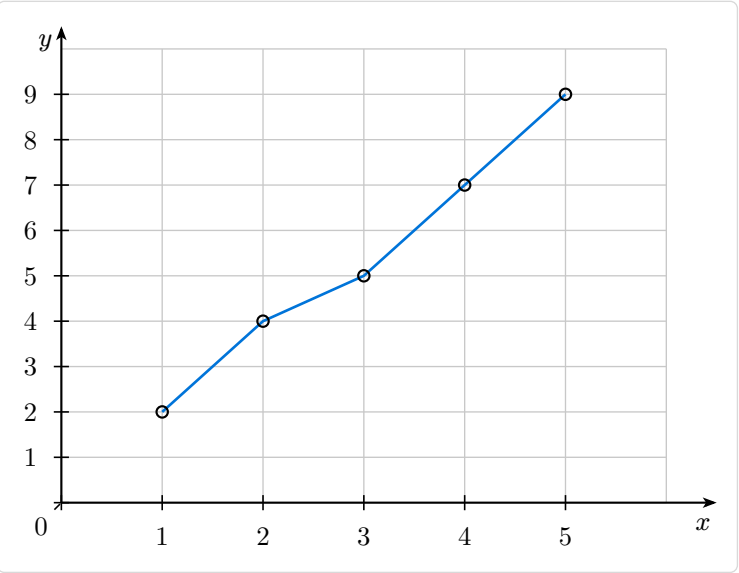
2.5. Data Points and Scatter Plots

Plot discrete data points:

Code

```
#plot(
  width: 8, height: 6,
  xmin: 0, xmax: 6, ymin: 0, ymax: 10,
  xlabel: $x$, ylabel: $y$,
  axis-x-pos: "bottom", axis-y-pos: "left",
  show-grid: "major",
  (
    data: ((1, 2), (2, 4), (3, 5), (4, 7), (5, 9)),
    mark: "o",
    mark-size: 0.15,
    stroke: blue + 1pt,
  ),
)
```

Preview



3. Grid Options

3.1. Grid Modes

Control grid display with `show-grid`:

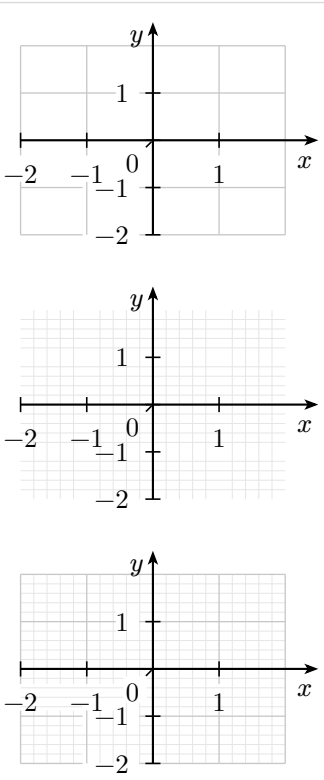
Code

```
// Major grid only
#plot(
  width: 5, height: 4,
  xmin: -2, xmax: 2, ymin: -2, ymax: 2,
  show-grid: "major",
)

// Minor grid only
#plot(
  width: 5, height: 4,
  xmin: -2, xmax: 2, ymin: -2, ymax: 2,
  show-grid: "minor",
)

// Both grids
#plot(
  width: 5, height: 4,
  xmin: -2, xmax: 2, ymin: -2, ymax: 2,
  show-grid: "both",
)
```

Preview



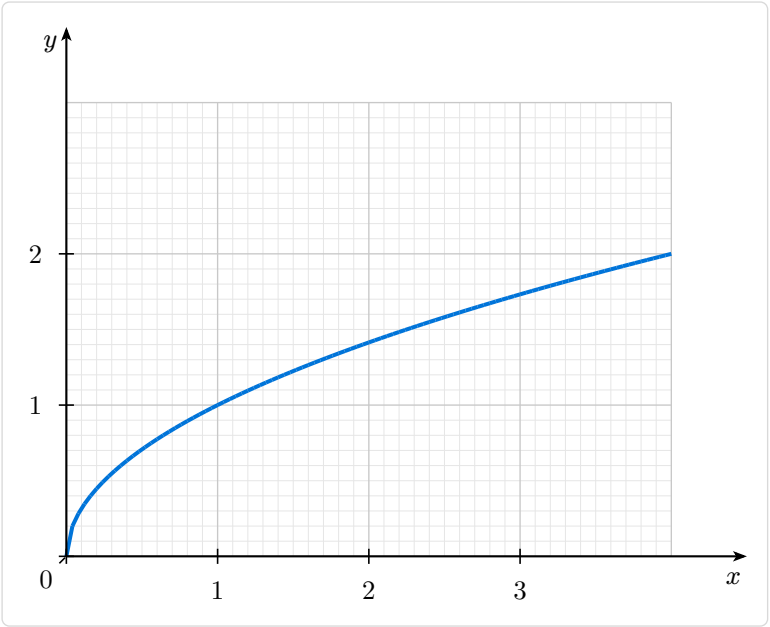
3.2. Minor Grid Subdivisions

Control the number of subdivisions with `minor-grid-step`:

Code

```
#plot(
  width: 8, height: 6,
  xmin: 0, xmax: 4, ymin: 0, ymax: 3,
  xlabel: $x$, ylabel: $y$,
  axis-x-pos: "bottom", axis-y-pos: "left",
  show-grid: "both",
  minor-grid-step: 10,  // 10 subdivisions per unit
  (fn: x => calc.sqrt(x), domain: (0, 4), stroke: blue + 1.5pt),
)
```

Preview



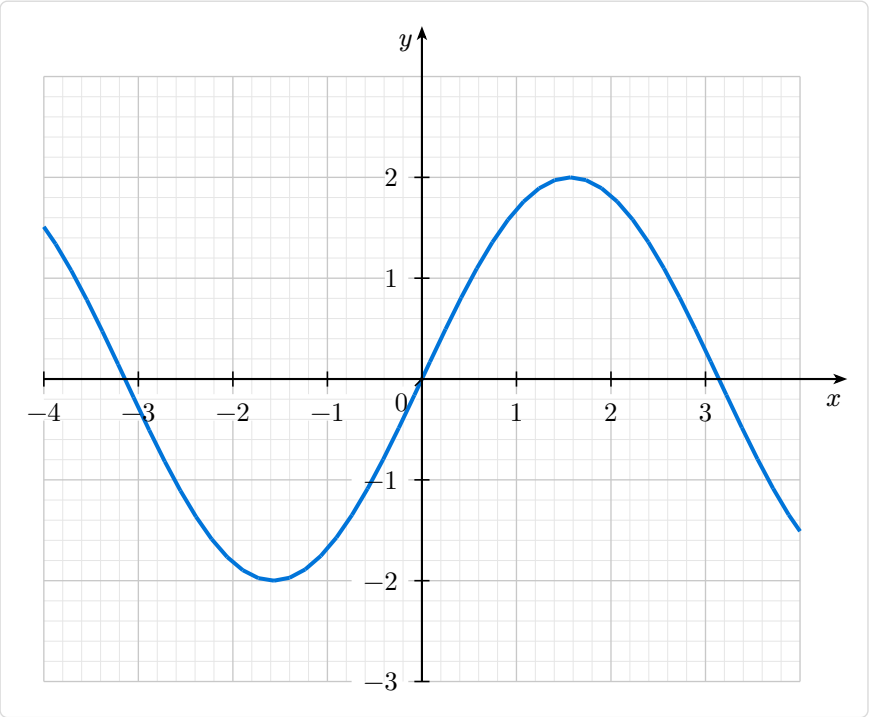
3.3. Grid Label Break

The `grid-label-break` option (enabled by default) draws grid lines with gaps around tick labels, creating an elegant break effect. Unlike a white-box approach, this works on any background color:

Code

```
#plot(
  width: 10, height: 8,
  xmin: -4, xmax: 4, ymin: -3, ymax: 3,
  xlabel: $x$, ylabel: $y$,
  show-grid: "both",
  minor-grid-step: 5,
  grid-label-break: true,  // Default
  (fn: x => calc.sin(x) * 2, stroke: blue + 1.5pt),
)
```

Preview



4. Axis Configuration

4.1. Axis Position

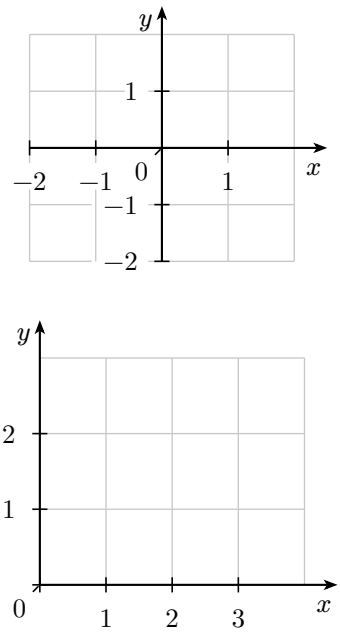
Position axes at origin (default), bottom/left, or custom values:

Code

```
// Through origin (default)
#plot(
  width: 5, height: 4,
  xmin: -2, xmax: 2, ymin: -2, ymax: 2,
  show-grid: "major",
)

// Bottom and left
#plot(
  width: 5, height: 4,
  xmin: 0, xmax: 4, ymin: 0, ymax: 3,
  axis-x-pos: "bottom",
  axis-y-pos: "left",
  show-grid: "major",
)
```

Preview



4.2. Axis Extension

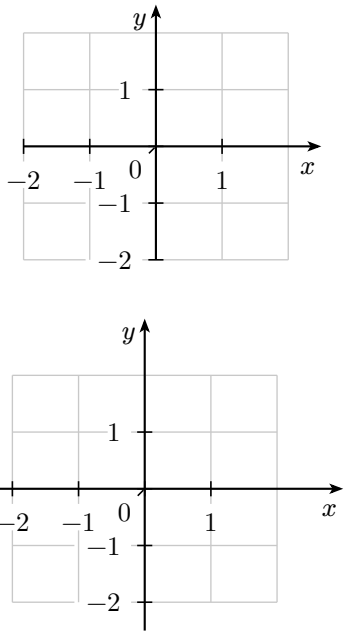
By default, axes extend 0.5 units beyond the grid on the arrow side. Customize with `axis-x-extend` and `axis-y-extend`:

Code

```
// Default extension (0, 0.5)
#plot(
  width: 5, height: 4,
  xmin: -2, xmax: 2, ymin: -2, ymax: 2,
  show-grid: "major",
)

// Custom extension
#plot(
  width: 5, height: 4,
  xmin: -2, xmax: 2, ymin: -2, ymax: 2,
  axis-x-extend: (0.5, 1),
  axis-y-extend: (0.5, 1),
  show-grid: "major",
)
```

Preview



5. Tick Configuration

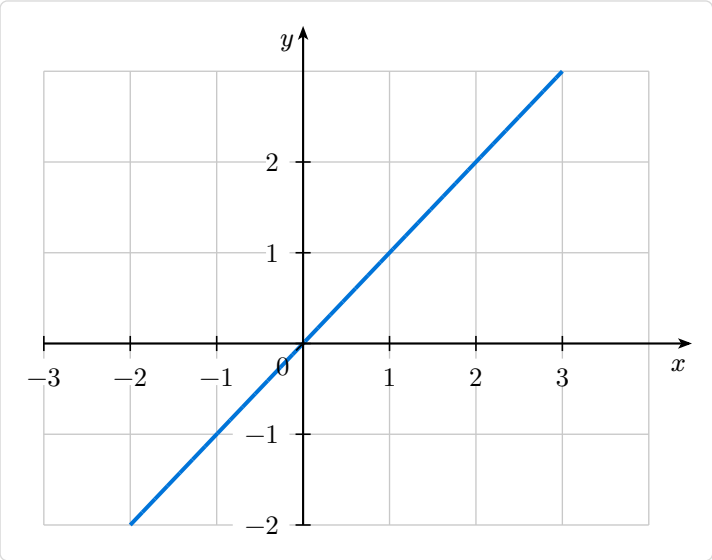
5.1. Default Integer Ticks

By default, ticks are placed at every integer (step = 1):

Code

```
#plot(
  width: 8, height: 6,
  xmin: -3, xmax: 4, ymin: -2, ymax: 3,
  xlabel: $x$, ylabel: $y$,
  show-grid: "major",
  (fn: x => x, stroke: blue + 1.5pt),
)
```

Preview



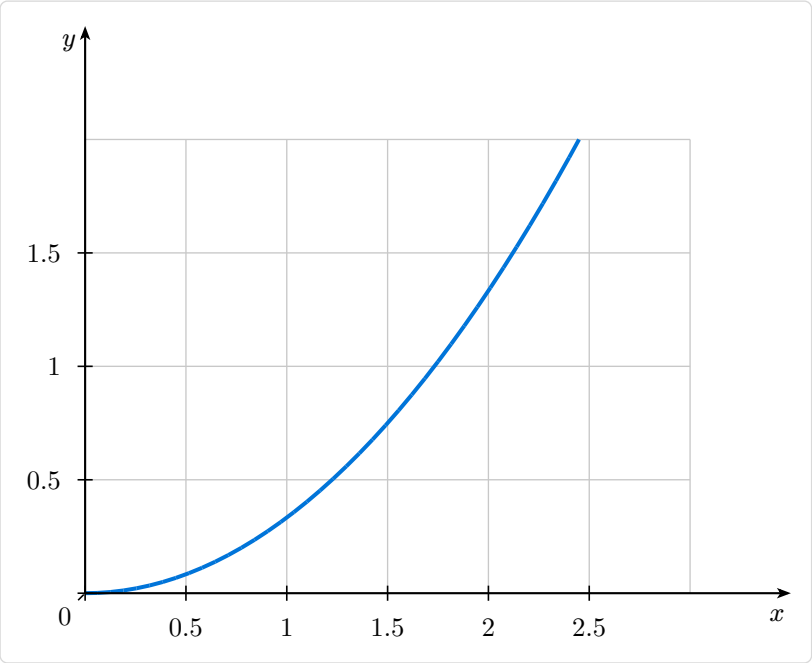
5.2. Custom Tick Step

Change tick spacing with `xtick-step` and `ytick-step`:

Code

```
#plot(
  width: 8, height: 6,
  xmin: 0, xmax: 3, ymin: 0, ymax: 2,
  xlabel: $x$, ylabel: $y$,
  axis-x-pos: "bottom", axis-y-pos: "left",
  xtick-step: 0.5,
  ytick-step: 0.5,
  show-grid: "major",
  (fn: x => x * x / 3, stroke: blue + 1.5pt),
)
```

Preview



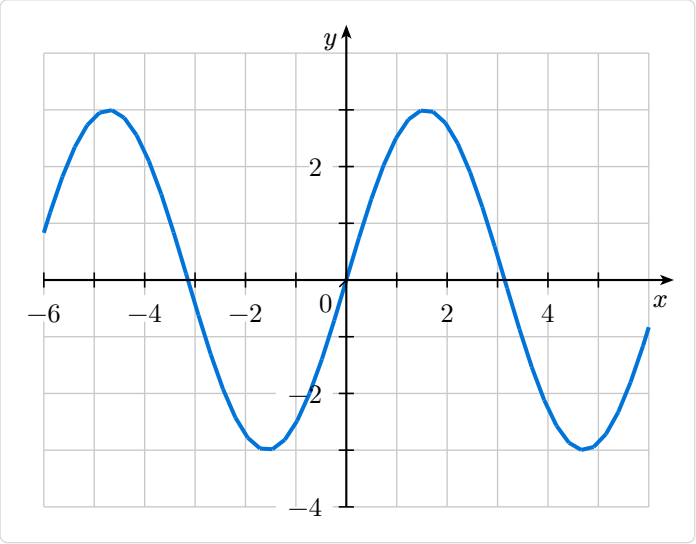
5.3. Tick Label Step

Show labels only at every N-th tick with `xtick-label-step` and `ytick-label-step`:

Code

```
#plot(
  width: 8, height: 6,
  xmin: -6, xmax: 6, ymin: -4, ymax: 4,
  xlabel: $x$, ylabel: $y$,
  xtick-label-step: 2,  // Labels at -6, -4, -2, 2, 4, 6
  ytick-label-step: 2,  // Labels at -4, -2, 2, 4
  show-grid: "major",
  (fn: x => calc.sin(x) * 3, stroke: blue + 1.5pt),
)
```

Preview



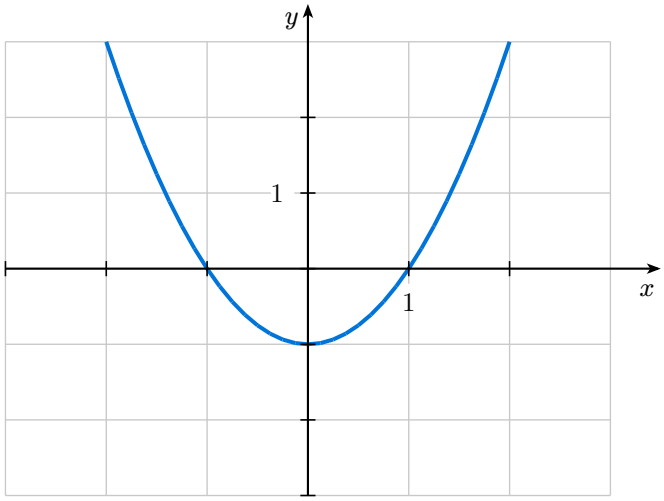
5.4. Unit Label Only

Show only “1” on each axis for a minimal style:

Code

```
#plot(
  width: 8, height: 6,
  xmin: -3, xmax: 3, ymin: -3, ymax: 3,
  xlabel: $x$, ylabel: $y$,
  unit-label-only: true,
  show-origin: false,
  show-grid: "major",
  (fn: x => x * x - 1, stroke: blue + 1.5pt),
)
```

Preview



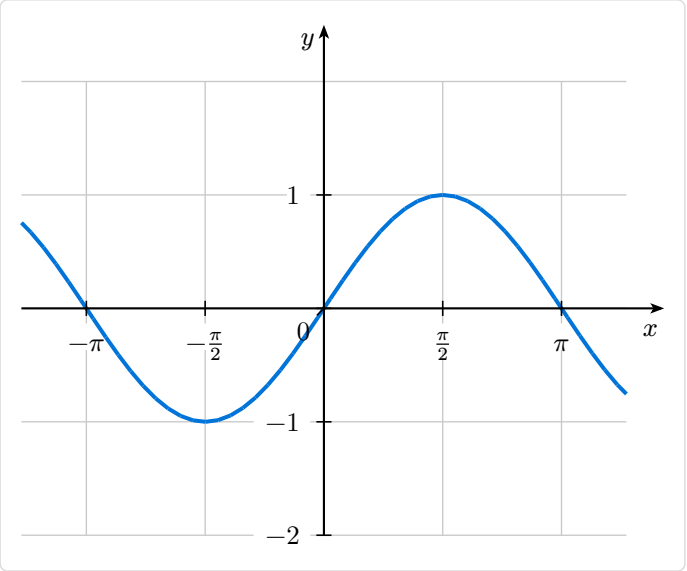
5.5. Custom Tick Positions

Specify exact tick positions with `xtick` and `ytick`:

Code

```
#plot(
  width: 8, height: 6,
  xmin: -4, xmax: 4, ymin: -2, ymax: 2,
  xlabel: $x$, ylabel: $y$,
  xtick: (-calc.pi, -calc.pi/2, 0, calc.pi/2, calc.pi),
  xtick-labels: ($-pi$, $-pi/2$, $0$, $pi/2$, $pi$),
  show-grid: "major",
  (fn: x => calc.sin(x), stroke: blue + 1.5pt),
)
```

Preview



5.6. Hide Origin Label

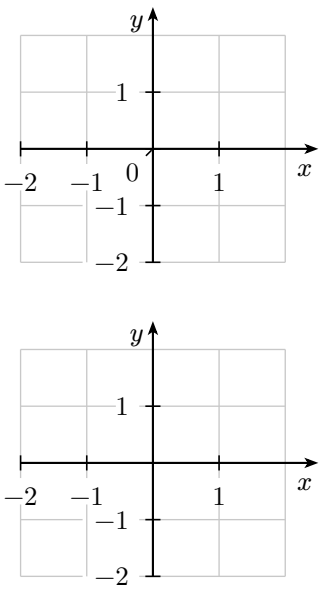
Control the “0” label at the origin:

Code

```
// With origin (default)
#plot(
  width: 5, height: 4,
  xmin: -2, xmax: 2, ymin: -2, ymax: 2,
  show-origin: true,
  show-grid: "major",
)

// Without origin
#plot(
  width: 5, height: 4,
  xmin: -2, xmax: 2, ymin: -2, ymax: 2,
  show-origin: false,
  show-grid: "major",
)
```

Preview



6. Markers

6.1. Available Marker Types

The following markers are available:

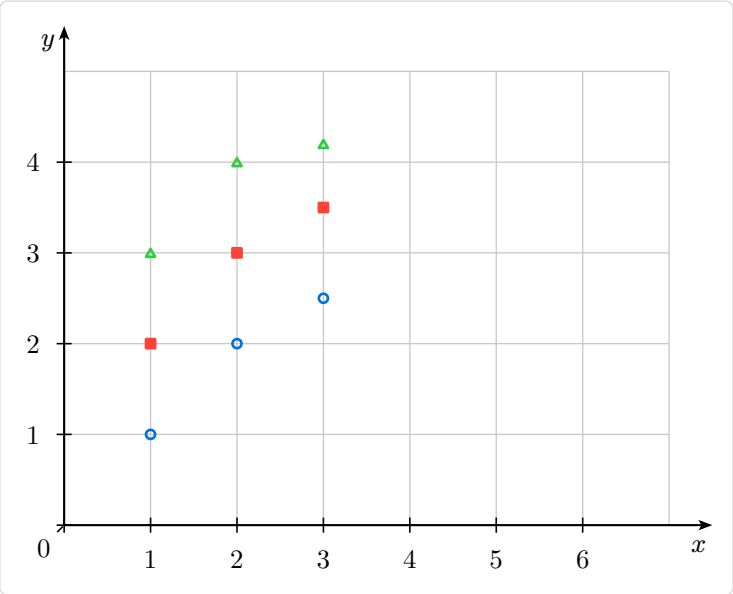
Marker	Description
`"o"`	Circle (outline)
`"*"`	Circle (filled)
`"square"`	Square (outline)
`"square*"`	Square (filled)
`"triangle"`	Triangle up (outline)
`"triangle*"`	Triangle up (filled)
`"diamond"`	Diamond (outline)
`"diamond*"`	Diamond (filled)
`"star"`	Star (outline)
`"star*"`	Star (filled)
`"+"`	Plus sign
`"x"`	Cross
`"|"`	Vertical bar
`"-"`	Horizontal bar

6.2. Using Markers

Code

```
#plot(
  width: 8, height: 6,
  xmin: 0, xmax: 7, ymin: 0, ymax: 5,
  axis-x-pos: "bottom", axis-y-pos: "left",
  show-grid: "major",
  data(((1, 1), (2, 2), (3, 2.5)), mark: "o", mark-stroke: blue),
  data(((1, 2), (2, 3), (3, 3.5)), mark: "square*", mark-fill: red, mark-stroke: red),
  data(((1, 3), (2, 4), (3, 4.2)), mark: "triangle", mark-stroke: green),
)
```

Preview



7. Convenience Functions

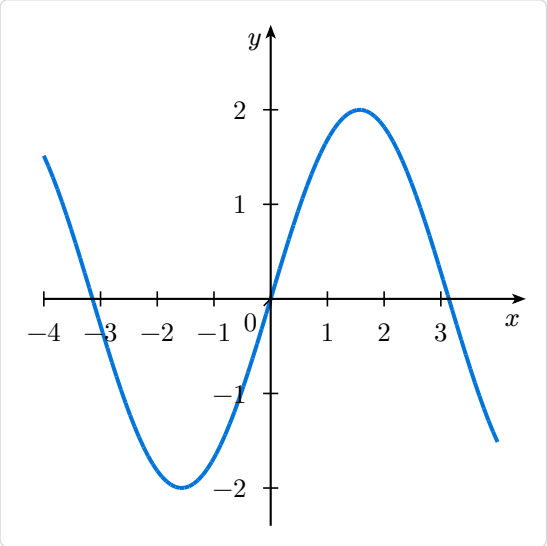
7.1. `plot-fn` - Quick Function Plot

Plot a single function with automatic y-scaling:

Code

```
#plot-fn(
  x => calc.sin(x) * 2.0,
  domain: (-4, 4),
  stroke: blue + 1.5pt,
)
```

Preview



Parameters:

Parameter	Type	Default	Description
`fn`	function	—	Function to plot
`domain`	array	(-5, 5)	X domain
`ymin` / `ymax`	float/auto	auto	Y bounds; `auto` = computed from samples
`stroke`	stroke	`blue + 1.2pt`	Line style
`..args`	any	—	Forwarded to `plot()`

7.2. `plot-rational` - Rational Function Wrapper

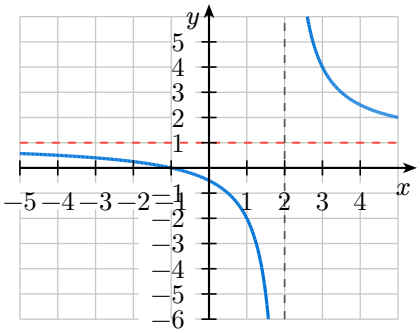
`plot-rational` is a convenience wrapper for common rational-function exercise graphs. It draws a single function on a centered coordinate grid and forwards extra arguments to `plot`, so you can add horizontal asymptotes, holes, or labels. Pass vertical asymptotes with `vertical-asymptotes`; the function domain is split around those values to avoid sampling at the pole.

```
#plot-rational(
  x => (x + 1) / (x - 2),
  xmin: -5, xmax: 5,
  ymin: -6, ymax: 6,
  vertical-asymptotes: (2,),
  hline(1, stroke: stroke(paint: red, thickness: 0.7pt, dash: "dashed")),
)
```

Code

```
#plot-rational(
  x => (x + 1) / (x - 2),
  xmin: -5, xmax: 5,
  ymin: -6, ymax: 6,
  vertical-asymptotes: (2,),
  hline(1, stroke: stroke(paint: red, thickness: 0.7pt,
dash: "dashed")),
)
```

Preview



Parameters:

Parameter	Type	Default	Description
`fn`	function	—	Function to plot
`xmin` / `xmax`	float	−6 / 6	X-axis bounds
`ymin` / `ymax`	float	−8 / 8	Y-axis bounds
`stroke`	stroke	`blue + 1.2pt`	Function stroke
`domain`	array/auto	auto	Function domain; `auto` = `(xmin, xmax)`
`samples`	int	200	Function sample count
`vertical-asymptotes`	array	()	X-values to skip and draw as vertical dashed lines
`asymptote-stroke`	stroke	dashed gray	Stroke for vertical asymptote lines
`asymptote-gap`	float	0.05	Half-gap removed around each vertical asymptote
`width` / `height`	float/auto	auto	Plot dimensions
`..args`	any	—	Forwarded to `plot()`; positional args become extra series

7.3. `limit-schema` / `schema-lim` - Local Limit Sketches

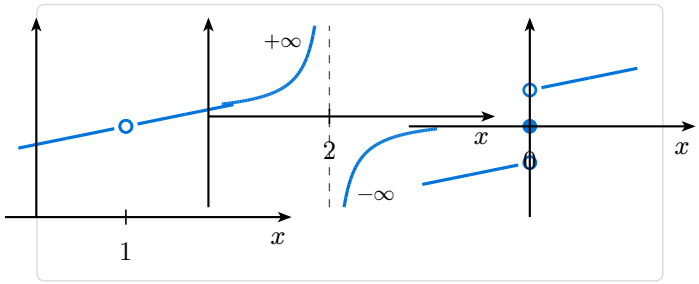
`limit-schema` draws a small schematic graph around a point `a`. It is useful for removable discontinuities, jumps, and vertical asymptotes. `schema-lim` is kept as a French alias.

```
#limit-schema(a: 1, left: 4, right: 4)
#limit-schema(a: 2, left: "+oo", right: "-oo")
#limit-schema(a: 0, left: -1, right: 1, val: 0)
```

Code

```
#grid(columns: (1fr, 1fr, 1fr), gutter: 0.5em,
  limit-schema(a: 1, left: 4, right: 4),
  limit-schema(a: 2, left: "+oo", right: "-oo"),
  limit-schema(a: 0, left: -1, right: 1, val: 0),
)
```

Preview



Parameters:

Parameter	Type	Default	Description
`a`	float	0	X-position of the local behavior
`left`	float/string/none	none	Left-hand behavior: finite value, `"+oo"`, `"-oo"`, or none
`right`	float/string/none	none	Right-hand behavior: finite value, `"+oo"`, `"-oo"`, or none
`val`	float/none	none	Defined value `f(a)` as a filled dot
`a-label`	content/auto	auto	Tick label at `a`; `auto` prints the value
`show-limit`	bool/float	false	Draw a dashed horizontal limit line; `true` auto-detects a common finite limit
`l-label`	content/auto	auto	Label for the limit line
`width` / `height`	float	3.8 / 2.8	Schema dimensions in cm

7.4. `scatter` / `data` - Point Sets

`scatter` and `data` are identical functions. Use them to create isolated point specifications. Set `connect: true` to join points with a line.

```
#plot(
  xmin: 0, xmax: 4, ymin: 0, ymax: 6,
  scatter(((1, 2), (2, 4), (3, 5)), mark: "o", mark-fill: blue),
  data(((1, 1.5), (2, 3.5), (3, 4.5)), mark: "*", mark-fill: red),
)
```

Parameters (both functions):

Parameter	Type	Default	Description
`points`	array	—	Array of `(x, y)` tuples
`mark`	string	`"*"`	Marker type
`mark-size`	float	0.12	Marker size in cm
`mark-fill`	color	blue	Marker fill color
`mark-stroke`	stroke	`blue + 0.8pt`	Marker stroke
`connect`	bool	false	Join points with a line
`stroke`	stroke	none	Line stroke when `connect: true`

`label`	content	none	Series label
`label-pos`	float	0.8	Position along series in [0, 1]
`label-anchor`	string	“south-west”	Label anchor

7.5. `line-plot` - Connected Line Plot

Create a line plot with markers (points joined by default):

```
#plot(
  xmin: 0, xmax: 4, ymin: 0, ymax: 6,
  line-plot(((1, 2), (2, 4), (3, 5)), stroke: blue + 1pt, mark: "o"),
)
```

Parameters:

Parameter	Type	Default	Description
`points`	array	—	Array of `(x, y)` tuples
`stroke`	stroke	`blue + 1.2pt`	Line stroke
`mark`	string	`"o"`	Marker type
`mark-size`	float	0.1	Marker size in cm
`mark-fill`	color	white	Marker fill
`mark-stroke`	stroke	`blue + 0.8pt`	Marker stroke
`label`	content	none	Series label
`label-pos`	float	0.8	Position along series in [0, 1]
`label-anchor`	string	“south-west”	Label anchor

7.6. `func-plot` - Function Plot Helper

Build a function series spec with full marker and label control:

```
#let my-func = func-plot(
  x => calc.sin(x),
  stroke: blue + 1.5pt,
  label: $sin(x)$,
  mark: "o",
  mark-interval: 15,  // marker every 15th sample
)
#plot(xmin: -4, xmax: 4, ymin: -1.5, ymax: 1.5, my-func)
```

Parameters:

Parameter	Type	Default	Description
`fn`	function	—	Function to plot
`domain`	array/auto	auto	X domain; `auto` = full plot range
`stroke`	stroke	`blue + 1.2pt`	Line style
`samples`	int	100	Sample count
`mark`	string	`"none"`	Marker type
`mark-size`	float	0.1	Marker size in cm

`mark-fill`	color	blue	Marker fill
`mark-stroke`	stroke	`blue + 0.8pt`	Marker stroke
`mark-interval`	int	10	Draw a marker every N-th sample
`label`	content	none	Curve label
`label-pos`	float	0.8	Position in [0, 1]
`label-anchor`	string	“south-west”	Label anchor

7.7. `parametric` - Ellipses and Hyperbolas

Draw curves that are not single-valued functions of `x`, such as ellipses:

```
#plot(
  xmin: -3, xmax: 3, ymin: -2, ymax: 2,
  parametric(t => 2 * calc.cos(t), t => calc.sin(t), domain: (0, 2 * calc.pi)),
)
```

For a hyperbola branch, use a finite parameter interval:

```
#plot(
  xmin: -4, xmax: 4, ymin: -3, ymax: 3,
  parametric(t => (calc.exp(t) + calc.exp(-t)) / 2, t => (calc.exp(t) - calc.exp(-t)) / 2,
domain: (-1.5, 1.5)),
  parametric(t => -(calc.exp(t) + calc.exp(-t)) / 2, t => (calc.exp(t) - calc.exp(-t)) / 2,
domain: (-1.5, 1.5)),
)
```

Parameters:

Parameter	Type	Default	Description
`fn-x`	function	—	X coordinate as function of t
`fn-y`	function	—	Y coordinate as function of t
`domain`	array	(0.0, 1.0)	Parameter range (t_1, t_2)
`stroke`	stroke	`blue + 1.2pt`	Curve stroke
`samples`	int	100	Number of parameter samples

7.8. `fill-area` - Fill Under a Curve

Fill the region between a function and a baseline (default: $y = 0$):

```
#plot(
  xmin: 0, xmax: calc.pi, ymin: -0.2, ymax: 1.2,
  fill-area(x => calc.sin(x), domain: (0, calc.pi), color: blue.lighten(70%)),
  (fn: x => calc.sin(x), domain: (0, calc.pi), stroke: blue + 1.5pt),
)
```

Use `baseline` for a non-zero base level, or `hatch` for hatched fills:

```
fill-area(x => calc.sin(x), domain: (0, calc.pi),
          hatch: "ne", hatch-spacing: 5pt, hatch-stroke: blue + 0.5pt)
```

Parameters:

Parameter	Type	Default	Description
`fn`	function	—	Function $f(x)$ bounding the top of the region
`domain`	array/auto	auto	X interval; `auto` = full plot range
`baseline`	float	0.0	Y-value of the bottom of the region
`color`	color	`luma(220)`	Fill color; `none` for hatch-only
`hatch`	string/none	none	Hatch pattern: `"ne"`, `"nw"`, `"h"`, `"v"`, `"cross"`, `"grid"`
`hatch-spacing`	length	`5pt`	Spacing between hatch lines
`hatch-stroke`	stroke	`luma(80) + 0.5pt`	Hatch line stroke
`samples`	int	80	Sample count

7.9. `area-between` - Fill Between Two Curves

Fill the region enclosed by two functions:

```
#plot(
  xmin: -1, xmax: 3, ymin: -1, ymax: 10,
  area-between(x => x * x, x => 3 * x, domain: (0, 3),
               color: green.lighten(60%)),
  (fn: x => x * x, stroke: blue + 1.5pt),
  (fn: x => 3 * x, stroke: red + 1.5pt),
)
```

Parameters:

Parameter	Type	Default	Description
`fn1`	function	—	First bounding function
`fn2`	function	—	Second bounding function
`domain`	array/auto	auto	X interval; `auto` = full plot range
`color`	color	`luma(220)`	Fill color; `none` for hatch-only
`hatch`	string/none	none	Hatch pattern
`hatch-spacing`	length	`5pt`	Hatch line spacing
`hatch-stroke`	stroke	`luma(80) + 0.5pt`	Hatch line stroke
`samples`	int	80	Sample count

7.10. `fill-closed` - Fill a Parametric Closed Curve

Fill the interior of a closed parametric curve (the curve should start and end at the same point):

```
#plot(
  xmin: -3, xmax: 3, ymin: -2, ymax: 2,
  fill-closed(t => 2 * calc.cos(t), t => calc.sin(t),
              domain: (0, 2 * calc.pi), color: blue.lighten(70%)),
  parametric(t => 2 * calc.cos(t), t => calc.sin(t),
             domain: (0, 2 * calc.pi), stroke: blue + 1.5pt),
)
```

Parameters:

Parameter	Type	Default	Description
`fn-x`	function	—	X coordinate as function of t
`fn-y`	function	—	Y coordinate as function of t
`domain`	array	(0.0, 1.0)	Parameter range; curve should be closed
`color`	color	`luma(220)`	Fill color; `none` for hatch-only
`hatch`	string/none	none	Hatch pattern
`hatch-spacing`	length	`5pt`	Hatch line spacing
`hatch-stroke`	stroke	`luma(80) + 0.5pt`	Hatch line stroke
`samples`	int	80	Sample count

7.11. `vline` / `hline` - Reference Lines

Draw vertical or horizontal reference lines at a fixed coordinate:

```
#plot(
  xmin: -3, xmax: 3, ymin: -2, ymax: 4,
  (fn: x => x * x - 1, stroke: blue + 1.5pt),
  vline(1.0, stroke: red + 0.8pt + (dash: "dashed")),
  hline(0.0, stroke: gray + 0.8pt),
)
```

Use `ymin`/`ymax` (for `vline`) or `xmin`/`xmax` (for `hline`) to draw partial lines:

```
vline(2.0, ymin: 0, ymax: 4)          // vertical from y=0 to y=4
hline(1.5, xmin: 0, xmax: 3)          // horizontal from x=0 to x=3
```

`vline` parameters:

Parameter	Type	Default	Description
`x0`	float	—	X position of the line
`stroke`	stroke	`luma(100) + 0.6pt`	Line stroke
`ymin`	float/auto	auto	Bottom y limit (default: plot `ymin`)
`ymax`	float/auto	auto	Top y limit (default: plot `ymax`)

`hline` parameters:

Parameter	Type	Default	Description
`y0`	float	—	Y position of the line
`stroke`	stroke	`luma(100) + 0.6pt`	Line stroke
`xmin`	float/auto	auto	Left x limit (default: plot `xmin`)
`xmax`	float/auto	auto	Right x limit (default: plot `xmax`)

7.12. `note` - Text Annotation

Place a text annotation at a data-coordinate position:

```
#plot(
  xmin: -3, xmax: 3, ymin: -2, ymax: 4,
  (fn: x => x * x - 1, stroke: blue + 1.5pt),
```

```
  note([$f(x) = x^2 - 1$], pos: (1.5, 2.5), anchor: "west", size: 9pt),
)
```

Parameters:

Parameter	Type	Default	Description
`body`	content	—	Annotation text or content
`pos`	array	—	`(x, y)` in data coordinates
`anchor`	string	“center”	Text anchor point
`size`	length	`9pt`	Font size

7.13. `riemann-sum` - Riemann Sum Rectangles

Draw left, right, midpoint, lower, or upper Riemann sum rectangles for a function. Use it as a plot item alongside the function curve or `fill-area`.

The five methods:

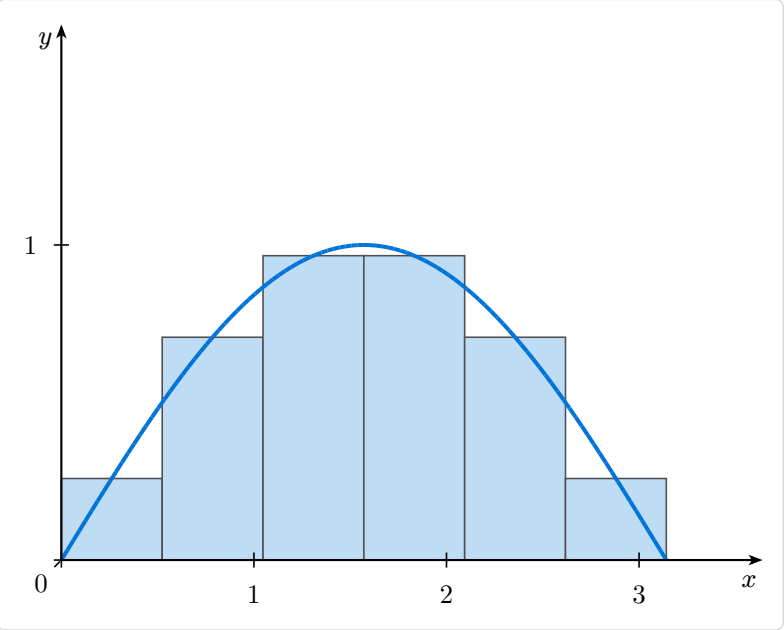
- `"left"` / `"right"` / `"mid"` — evaluation at the left endpoint, right endpoint, or midpoint of each sub-interval
- `"lower"` / `"upper"` — true infimum / supremum sampled within each sub-interval; correct for any function shape including U-curves

7.13.1. Basic Example

Code

```
#plot(
  width: 8, height: 5,
  xmin: 0, xmax: calc.pi, ymin: 0, ymax: 1.2,
  axis-x-pos: "bottom", axis-y-pos: "left",
  riemann-sum(
    x => calc.sin(x),
    domain: (0.0, calc.pi),
    n: 6, method: "mid",
    color: blue.lighten(75%),
  ),
  (fn: x => calc.sin(x), domain: (0.0, calc.pi), stroke: blue + 1.5pt),
)
```

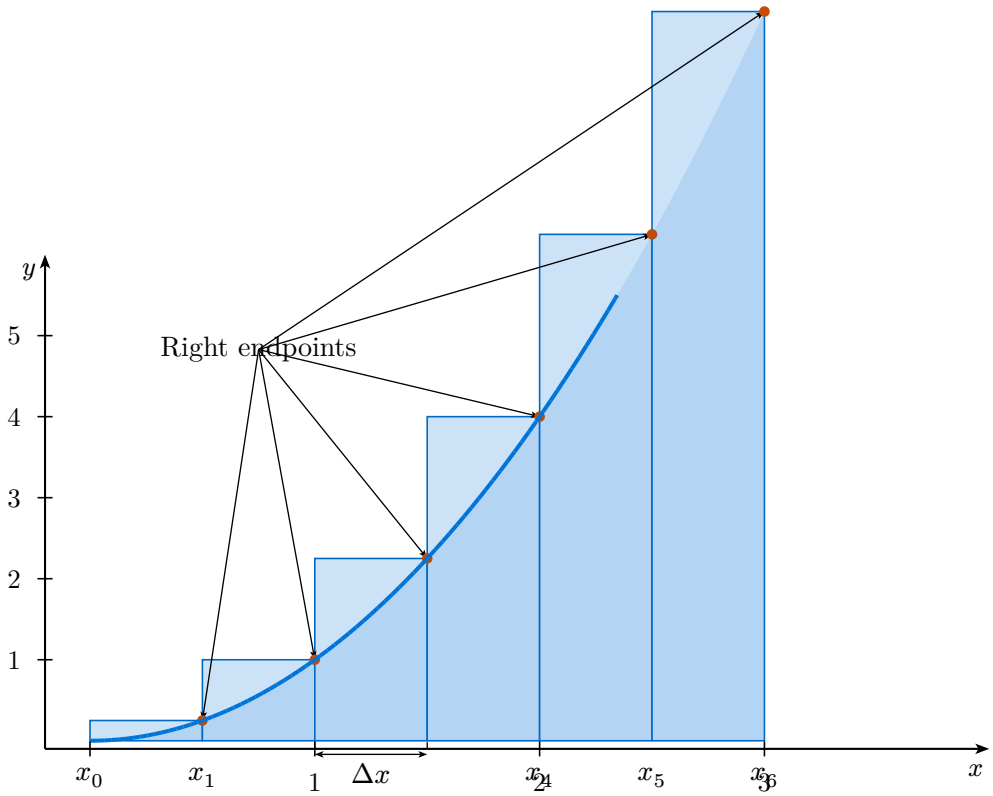
Preview



7.13.2. Annotations: `show-points`, `show-dx`, `show-xi`

Turn on the annotation overlays for pedagogical diagrams:

```
riemann-sum(
  x => x * x,
  domain: (0.0, 3.0),
  n: 6, method: "right",
  color: blue.lighten(80%),
  show-points: true,           // dots at evaluation points with arrows
  show-dx: true, dx-rect: 2,   // Δx bracket under rectangle 2
  show-xi: true,               // x₀ … x₆ labels along the axis
)
```



7.13.3. `xi-show-values`

Set `xi-show-values: true` to display the actual numeric values of the subdivision points instead of the symbolic x_i notation:

```
riemann-sum(fn, domain: (0.0, 4.0), n: 4, method: "left",
            show-xi: true, xi-show-values: true)
```

7.13.4. Hatch patterns

Use `hatch` together with `hatch-spacing` and `hatch-stroke` to draw hatched rectangles. Set `color: none` for hatch-only (no fill):

```
riemann-sum(fn, domain: ..., n: 6, method: "mid",
            color: none,
            hatch: "ne", hatch-spacing: 4pt, hatch-stroke: blue + 0.6pt)
```

Available hatch styles: `"ne"`, `"nw"`, `"h"`, `"v"`, `"cross"`, `"grid"`.

7.14. `volume-of-revolution` - Volume of revolution diagram

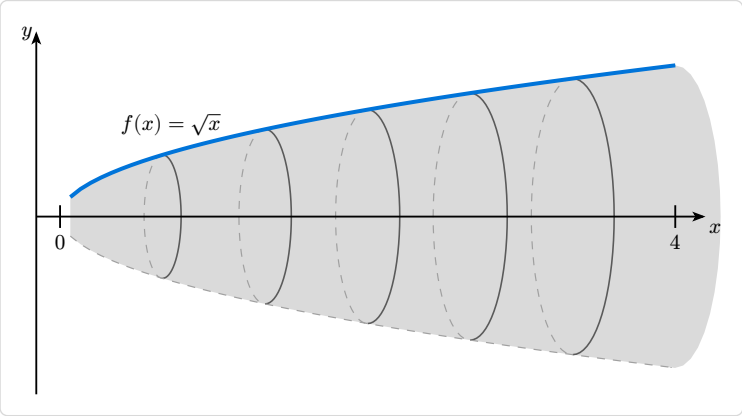
Draw a 3D-style solid generated by rotating a profile $y = f(x)$ around a horizontal or oblique axis. The renderer draws the filled body, front and back cap ellipses, intermediate disk cross-sections, axis arrow, coordinate y-axis, and optional labels.

7.14.1. Basic Example

Code

```
#volume-of-revolution(
  x => calc.sqrt(x),
  domain: (0.0, 4.0),
  n-disks: 5,
  width: 8.0, height: 4.0,
  show-yaxis: true,
  label-a: $0$, label-b: $4$,
  label-f: $f(x)=sqrt(x)$,
)
```

Preview



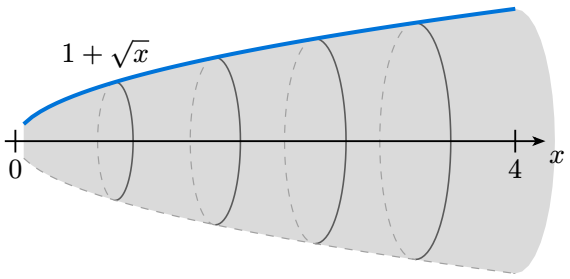
7.14.2. Axis of Revolution

Use `axis-y` to revolve around $y = c$ (shifted horizontal axis), or `axis-slope` for an oblique axis $y = mx +$ axis-y:

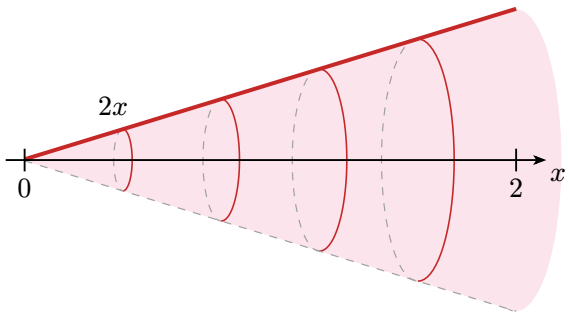
```
// Around y = 1
#volume-of-revolution(x => 1.0 + calc.sqrt(x), domain: (0, 4), axis-y: 1.0)

// Around y = x (oblique)
#volume-of-revolution(x => x * x, domain: (0.01, 2), axis-slope: 1.0)
```

Around $y = 1$



Around $y = x$ (oblique)



7.14.3. `show-back` and `show-radius-marker`

`show-back: false` hides the dashed back half and bottom profile — useful for side-by-side comparisons. `show-radius-marker: true` draws a vertical dimension marker at the y-axis:

```
#volume-of-revolution(fn, domain: ...,
  show-back: false,
  show-radius-marker: true,
  yaxis-x: 2.0,   // place y-axis at x=2, not the auto left position
  label-y: $sqrt(2)$,
)
```

7.14.4. `show-y-axis` and y-axis placement

The coordinate y-axis is positioned automatically to the left of the solid by default. Use `y-axis-x` / `yaxis-x` to fix it at a specific x-value, `y-axis-offset` to change the automatic gap, and `y-axis-extend` to control how far the axis extends above and below the solid:

```
#volume-of-revolution(fn, domain: ...,
  show-yaxis: true,
  y-axis-offset: 0.6,           // more gap from the volume
  y-axis-extend: (0.2, 0.5),   // tighter below, more above
)
```

8. Global Configuration

8.1. Setting Defaults

Use `set-plot-defaults` to configure defaults for all subsequent plots. Any `plot` parameter can be used, including `style`:

```
#set-plot-defaults(
  width: 8,
  height: 6,
  show-grid: "both",
  minor-grid-step: 5,
  style: (
    axis: (stroke: navy + 0.8pt, arrow: (symbol: "stealth", fill: navy, scale: 0.55)),
    ticks: (stroke: navy + 0.6pt, label-fill: navy),
  ),
)

// All subsequent plots will use these defaults
#plot(xmin: -3, xmax: 3, ymin: -2, ymax: 2, ...)
```

Per-call `style:` arguments are merged on top of the default style, so you can override individual properties without restating the full style.

8.2. Resetting Defaults

Reset to original defaults:

```
#reset-plot-defaults()
```

9. Styling

9.1. Custom Styles

Override default styles with the `style` parameter:

```
#plot(
  xmin: -3, xmax: 3, ymin: -2, ymax: 2,
  style: (
    background: (fill: rgb("#202124")),
    axis: (
      stroke: white + 1pt,
      arrow: (symbol: "stealth", fill: white, scale: 0.55),
    ),
    grid: (
      major: (stroke: luma(120) + 0.6pt),
      minor: (stroke: luma(80) + 0.3pt),
    ),
    ticks: (
      length: 0.12,
      stroke: white + 0.6pt,
      label-size: 0.7em,
      label-fill: white,
    ),
    labels: (
      fill: white,
    ),
  ),
  ...
)
```

9.2. Default Style Values

Property	Default	Description
`background.fill`	`none`	Plot background fill
`background.stroke`	`none`	Plot background stroke
`axis.stroke`	`black + 0.8pt`	Axis line style
`axis.arrow`	`(symbol: "stealth", fill: black, scale: 0.55)`	Arrow head style
`grid.major.stroke`	`luma(200) + 0.5pt`	Major grid line style
`grid.minor.stroke`	`luma(230) + 0.3pt`	Minor grid line style
`ticks.length`	`0.1`	Tick mark length (cm)
`ticks.stroke`	`black + 0.6pt`	Tick mark style
`ticks.label-size`	`0.65em`	Tick label font size
`ticks.label-fill`	`black`	Tick label color
`ticks.label-offset`	`0.15`	Distance from tick to label
`plot.stroke`	`blue + 1.2pt`	Default function stroke
`plot.samples`	`100`	Default sample count
`marker.size`	`0.12`	Default marker size
`labels.size`	`0.8em`	Axis label font size

`labels.fill`	`black`	Axis label color

10. Parameter Reference

10.1. `plot` Function

Dimensions and Bounds:

Parameter	Type	Default	Description
`width`	float	6	Plot width in cm
`height`	float	6	Plot height in cm
`scale`	float	1	Scale factor for entire plot
`xmin`	float	-5	Minimum x value
`xmax`	float	5	Maximum x value
`ymin`	float	-5	Minimum y value
`ymax`	float	5	Maximum y value

Axis Configuration:

Parameter	Type	Default	Description
`xlabel`	content	x	X-axis label
`ylabel`	content	y	Y-axis label
`xlabel-pos`	string/array	“end”	“end”, “center”, or (x, y)
`ylabel-pos`	string/array	“end”	“end”, “center”, or (x, y)
`xlabel-anchor`	string	“north”	Anchor for x label
`ylabel-anchor`	string	“east”	Anchor for y label
`xlabel-offset`	array	(0.0, -0.05)	X label offset (cm)
`ylabel-offset`	array	(-0.05, 0.0)	Y label offset (cm)
`axis-x-pos`	string/float	0	“bottom”, “center”, or y-value
`axis-y-pos`	string/float	0	“left”, “center”, or x-value
`axis-x-extend`	float/array	(0, 0.5)	X-axis extension (left, right)
`axis-y-extend`	float/array	(0, 0.5)	Y-axis extension (bottom, top)

Tick Configuration:

Parameter	Type	Default	Description
`xtick`	array/none	auto	Custom x tick positions
`ytick`	array/none	auto	Custom y tick positions
`xtick-step`	float	1	X tick spacing
`ytick-step`	float	1	Y tick spacing
`xtick-labels`	array/none	auto	Custom x tick labels
`ytick-labels`	array/none	auto	Custom y tick labels
`xtick-label-step`	int	1	Show x label every N ticks
`ytick-label-step`	int	1	Show y label every N ticks

`show-origin`	bool	true	Show “0” at origin
`origin-label-offset`	array	(-0.11, −0.11)	Origin “0” label offset from (0,0) in cm
`origin-label-anchor`	string	“north-east”	Origin “0” label anchor
`origin-leader`	bool	true	Draw a subtle leader line from origin label toward (0,0)
`origin-leader-stroke`	stroke	`black + 0.6pt`	Origin leader stroke
`origin-leader-gap`	float	0.025	Gap from (0,0) before leader starts (cm)
`origin-leader-end-gap`	float	0.025	Gap before the label anchor (cm)
`unit-label-only`	bool	false	Show only “1” on axes
`tick-label-size`	length	0.65em	Tick label font size
`axis-label-size`	length	0.8em	Axis label font size

Grid Configuration:

Parameter	Type	Default	Description
`show-grid`	bool/string	false	true, false, “major”, “minor”, “both”
`minor-grid-step`	int	5	Subdivisions per major tick
`grid-label-break`	bool	true	Break grid lines around labels

Styling:

Parameter	Type	Default	Description
`style`	dictionary	none	Style overrides (see Styling section)
`series`	array	none	Pre-built array of series specs (alternative to positional args)

10.2. Function/Data Specification

Each plot item is a dictionary with these fields:

Field	Type	Description
`fn`	function	Function to plot: `x => y` (return `none` to create a hole/discontinuity)
`data`	array	Data points: `((x1, y1), (x2, y2), ...)`
`connect`	bool	Connect data points with a line (default: `true` for `fn`, `false` for `data`)
`domain`	array	Function domain: `(xmin, xmax)`
`samples`	int	Number of samples for function (default: 100)
`stroke`	stroke	Line style (default: `blue + 1.2pt`)
`mark`	string	Marker type (default: `"none"`)
`mark-size`	float	Marker size in cm (default: 0.12)
`mark-fill`	color	Marker fill color (default: black)
`mark-stroke`	stroke	Marker stroke style (default: `black + 0.8pt`)
`label`	content	Label text placed near the curve
`label-pos`	float	Position along the curve in [0, 1] (default: 1.0)
`label-side`	string	Label placement relative to curve: `"above"`, `"below"`, `"left"`, `"right"`, `"above-left"`, etc.
`label-anchor`	string	CeTZ anchor override (alternative to `label-side`)

10.3. `riemann-sum` Function

Parameter	Type	Default	Description
`fn`	function	—	Function $f(x)$ to approximate
`domain`	array	plot range	Interval `(a, b)`; overrides the plot's x-range
`n`	int	4	Number of rectangles
`method`	string	`"right"`	`"left"`, `"right"`, `"mid"`, `"lower"`, `"upper"`
`baseline`	float	0.0	Y-level of rectangle bases (default: x-axis)
`color`	color	`luma(220)`	Rectangle fill; `none` for hatch-only
`stroke`	stroke	`luma(80) + 0.6pt`	Rectangle border stroke
`hatch`	string/none	none	Hatch pattern: `"ne"`, `"nw"`, `"h"`, `"v"`, `"cross"`, `"grid"`
`hatch-spacing`	length	`5pt`	Spacing between hatch lines
`hatch-stroke`	stroke	`luma(80) + 0.5pt`	Stroke for hatch lines
`samples`	int	20	Samples per sub-interval for `"lower"` / `"upper"` methods
`show-points`	bool	false	Draw a dot at each evaluation point
`point-color`	color	`rgb("#c94a00")`	Dot fill color
`point-size`	float	0.07	Dot radius in cm
`point-label`	content/auto/none	`auto`	Arrow label near dots; `auto` = method name, `none` = dots only
`point-label-pos`	array/auto	`auto`	`(x, y)` in data coords; `auto` = upper-right of dots
`show-dx`	bool	false	Draw a Δx dimension bracket under one rectangle
`dx-rect`	int/auto	`auto`	Rectangle to annotate (0-based); `auto` = middle rectangle
`dx-label`	content	`$Delta x$`	Label displayed inside the bracket
`show-xi`	bool	false	Draw $x_0, x_1, l..., x_n$ at subdivision points
`xi-labels`	array/auto	`auto`	Custom label array; `auto` = subscripted x_i notation
`xi-show-values`	bool	false	Show numeric x-values instead of x_i notation

10.4. `volume-of-revolution` Function

`solid-of-revolution` is an alias kept for backward compatibility.

Parameter	Type	Default	Description
`fn`	function	—	Profile function $f(x)$; the curve that is revolved
`domain`	array	`(0.0, 4.0)`	Interval `(a, b)` — the revolution interval
`n-disks`	int	4	Number of intermediate disk cross-sections to show
`width` / `height`	float	5.0 / 3.5	Canvas size in cm
`samples`	int	60	Number of points sampled along the profile curve
`axis-y`	float	0.0	Y-value of horizontal revolution axis ($y = c$)
`axis-slope`	float	0.0	Slope m: revolution axis is $y = mx +$ axis-y
`show-axis`	bool	true	Draw the revolution axis with an arrowhead
`show-y-axis` / `show-yaxis`	bool	false	Draw a coordinate y-axis at the left cap
`y-axis-x` / `yaxis-x`	float/auto	`auto`	X-position for the y-axis; `auto` = left of volume
`y-axis-offset`	float	0.45	Gap between y-axis and the volume when `auto`
`y-axis-extend`	array	`(0.35, 0.45)`	Y-axis padding `(below, above)` the solid
`show-radius-marker`	bool	false	Draw a vertical radius dimension marker at `yaxis-x`
`show-back`	bool	true	Show the back half of the solid (dashed ellipses + bottom curve)
`show-labels`	bool	true	Show a, b, f labels
`profile-stroke`	stroke	`blue + 1.5pt`	Top profile curve stroke
`disk-color`	color	`luma(218)`	Solid body fill color
`disk-stroke`	stroke	`luma(90) + 0.6pt`	Disk edge stroke
`axis-stroke`	stroke	`black + 0.7pt`	Revolution axis stroke
`label-a` / `label-b`	content	`$a$` / `$b$`	Labels at domain endpoints
`label-f`	content	`$f$`	Function label near the profile curve
`label-y`	content	`$y$`	Label for the coordinate y-axis

10.5. `zoom` - Spy / zoom inset

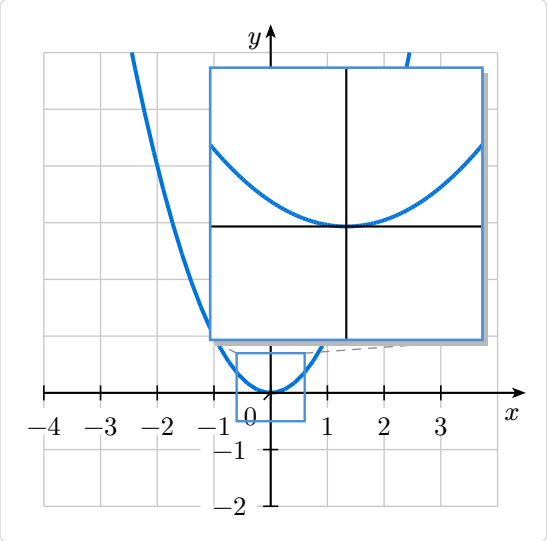
Add a magnified sub-view (spy glass) of any region of the main plot. The `zoom()` helper returns a series spec that is passed directly to `plot()`. It draws a highlighted region on the main plot and a zoomed inset box with connector lines.

10.5.1. Basic Example

Code

```
#plot(
  xmin: -4, xmax: 4, ymin: -2, ymax: 6,
  show-grid: "major",
  (fn: x => x * x, stroke: blue + 1.5pt),
  zoom(center: (0, 0.1), size: 0.9, magnification: 4, at: "top-right"),
)
```

Preview

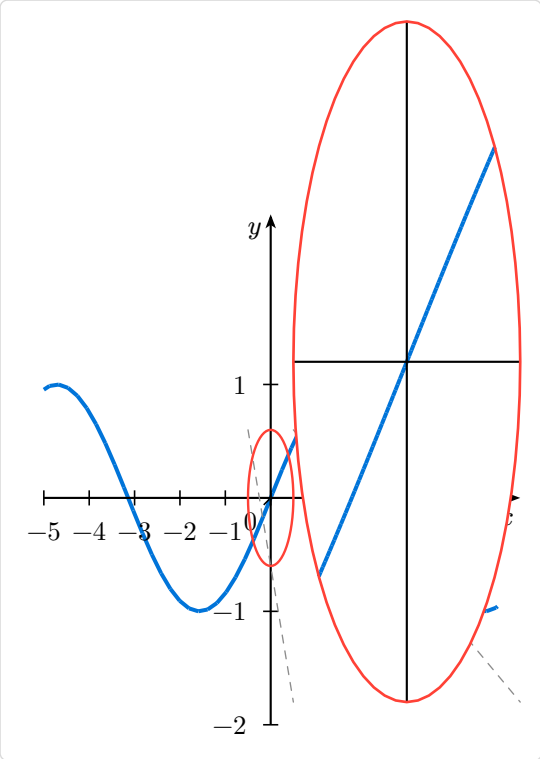


10.5.2. Circular lens, explicit region

Code

```
#plot(
  xmin: -5, xmax: 5, ymin: -2, ymax: 2,
  (fn: x => calc.sin(x), stroke: blue + 1.5pt),
  zoom(region: (-0.5, -0.6, 0.5, 0.6),
       lens-shape: "circle", magnification: 5,
       at: (3, 1.2), accent: red),
)
```

Preview



10.5.3. Parameters

Parameter	Type	Default	Description
`region`	array	`auto`	Spy glass corners `(x1, y1, x2, y2)` in data coords. Either `region` or `center` + `size` is required.
`center`	array	`auto`	Spy glass center `(cx, cy)` in data coords (use with `size`)
`size`	float	`auto`	Spy glass size in cm — ensures a square glass regardless of axis ratio
`at`	array/str	`auto`	Inset box center: `(x,y)` data coords, or a placement keyword: `"top-right"`, `"top-left"`, `"bottom-right"`, `"bottom-left"`, `"top"`, `"bottom"`, `"left"`, `"right"`. `auto` = smart opposite-quadrant placement
`width` / `height`	float	`auto`	Inset box size in cm. `auto` = derived from `magnification` or 3.5
`magnification`	float	`auto`	Zoom factor — inset size = spy glass canvas size × factor
`lens-shape`	str	`"rect"`	`"rect"` or `"circle"`
`connect`	bool	true	Draw connector lines between spy glass and inset
`accent`	color	blue	Accent color for borders and region highlight
`region-fill`	color	`auto`	Fill tint inside the spy glass region

`region-stroke`	stroke	`auto`	Border of the spy glass region
`box-fill`	color	white	Background fill of the inset box
`box-stroke`	stroke	`auto`	Border of the inset box
`connector-stroke`	stroke	`auto`	Dashed gray connector lines by default
`connector-fill`	color	none	Fill the trapezoid between connector lines
`shadow`	bool	`auto`	Drop shadow behind the inset (default true for rect with fill)
`show-inset-grid`	bool	true	Subtle grid inside the inset aligned to main ticks
`show-magnification`	bool	false	Show ×N label in inset corner
`label`	content	none	Custom label inside the inset (overrides the ×N label)

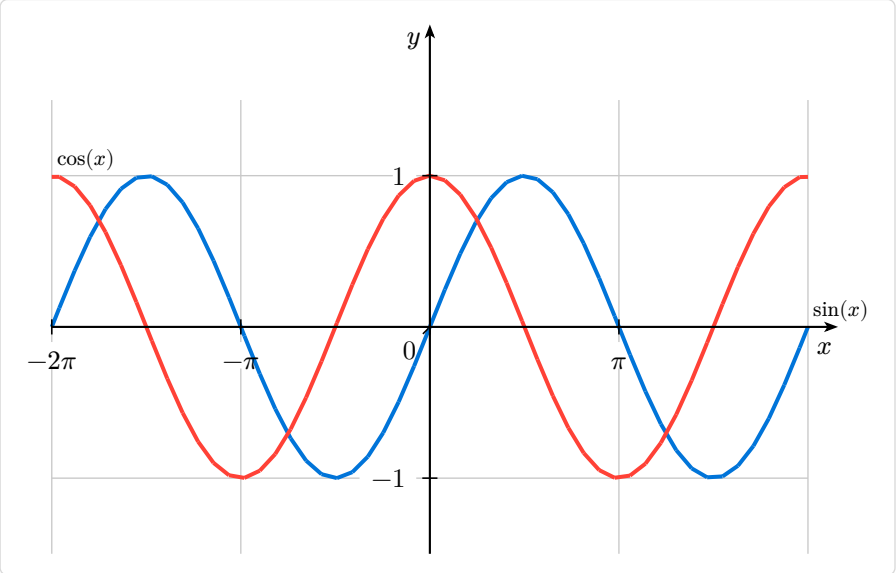
11. Complete Examples

11.1. Trigonometric Functions

Code

```
#plot(
  width: 10, height: 6,
  xmin: -2 * calc.pi, xmax: 2 * calc.pi,
  ymin: -1.5, ymax: 1.5,
  xlabel: $x$, ylabel: $y$,
  xtick: (-2*calc.pi, -calc.pi, 0, calc.pi, 2*calc.pi),
  xtick-labels: ($-2pi$, $-pi$, $0$, $pi$, $2pi$),
  show-grid: "major",
  (fn: x => calc.sin(x), stroke: blue + 1.5pt, label: $sin(x)$, label-pos: 1),
  (fn: x => calc.cos(x), stroke: red + 1.5pt, label: $cos(x)$, label-pos: 0),
)
```

Preview

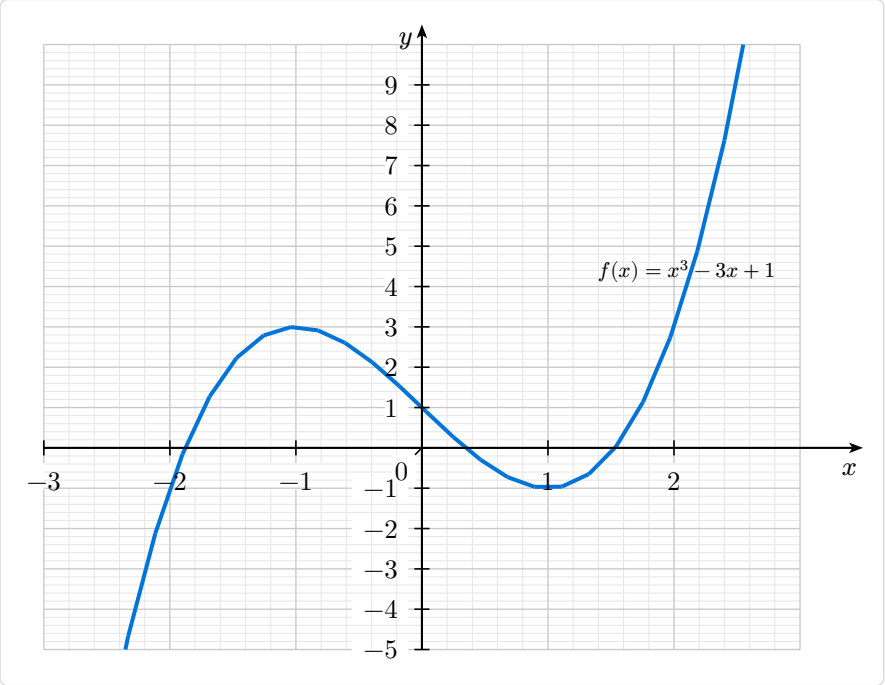


11.2. Polynomial with Fine Grid

Code

```
#plot(
  width: 10, height: 8,
  xmin: -3, xmax: 3, ymin: -5, ymax: 10,
  xlabel: $x$, ylabel: $y$,
  show-grid: "both",
  minor-grid-step: 5,
  (
    fn: x => x * x * x - 3 * x + 1,
    stroke: blue + 1.5pt,
    label: $f(x) = x^3 - 3x + 1$,
    label-side: "above",
    label-pos: 0.85,
  ),
)
```

Preview

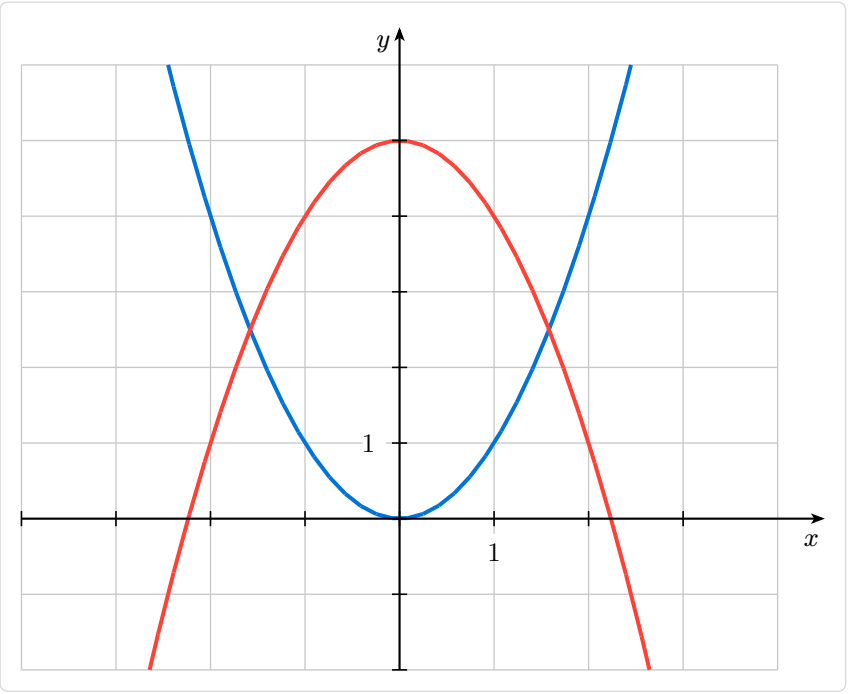


11.3. Minimal Style Plot

Code

```
#plot(
  width: 10, height: 8,
  xmin: -4, xmax: 4, ymin: -2, ymax: 6,
  xlabel: $x$, ylabel: $y$,
  show-grid: "major",
  unit-label-only: true,
  show-origin: false,
  (fn: x => x * x, stroke: blue + 1.5pt),
  (fn: x => -x * x + 5, stroke: red + 1.5pt),
)
```

Preview



11.4. Data with Trend Line

Code

```
#plot(
  width: 10, height: 7,
  xmin: 0, xmax: 6, ymin: 0, ymax: 12,
  xlabel: "Time (s)", ylabel: "Distance (m)",
  axis-x-pos: "bottom", axis-y-pos: "left",
  show-grid: "both",
  minor-grid-step: 5,
  (fn: x => 2 * x, stroke: gray + 1pt, domain: (0, 6)),  // Trend line
  (
    data: ((0.5, 1.2), (1, 2.3), (2, 3.8), (3, 6.2), (4, 7.9), (5, 10.1)),
    mark: "o",
    mark-size: 0.12,
    stroke: none,
  ),
)
```

Preview